Exchange de Criptomoedas

Documentação do projeto

CCP230 - Desenvolvimento de Algoritmos

Gabriel Marinov Baraglio

Centro Universitário FEI

1. Introdução

Esse é o meu projeto de Exchange de Criptomoedas, desenvolvido durante a disciplina de Desenvolvimento de Algoritmos (Linguagem C) no 2º período de Ciência da Computação na FEI. Além de buscar escrever um bom código, procurei organizar e modularizar tudo da melhor maneira possível. Também me preocupei em otimizar o uso de memória e a performance, utilizando definições no header em vez de declarar variáveis diretamente em alguns cenários.

Todo o conhecimento aplicado aqui foi adquirido nas aulas e através de estudos por conta própria, principalmente no W3Schools [https://www.w3schools.com/]. Outra base importante de aprendizado foram minhas contribuições em projetos open-source, como no LineageOS [https://github.com/GMB8281]. Nele, fiz adaptações para trazer versões mais novas do Android a celulares e tablets incompatíveis, o que me deu bastante contato prático com C/C++ direto na ponta dos drivers, além de Java e Kotlin na camada do sistema (até a data de entrega deste projeto, ainda não publiquei todos esses códigos, mas lançarei assim que estiverem totalmente estáveis).

2. Estrutura do projeto

O código-fonte fica em `src/`, dividido em quatro módulos (um por pasta) mais o arquivo `run.c`, que contém a função `main`. Os três arquivos `.csv` da raiz são criados automaticamente na primeira execução e estão listados no `.gitignore` (usei git para armazenar e transportar código durante o desenvolvimento).

```
ProjetoC_CCP230_SP_Exchange_Criptomoedas/
    build-linux.sh
    build-windows.bat
    LeiaMe.md
    LICENSE
    .gitignore
    src/
        run.c
        dados/
            dados.h
            memoria.c
            arquivos.c
            carrega_dados.c
            salva_dados.c
        login/
            login.h
            login_cadastro.c
        menu/
            menu.h
            entrada.c
            menu_principal.c
            submenu.c
        moedas/
            moedas.h
            compra_venda.c
            cotacao.c
            saldo_extrato.c
```

3. Como rodar?

O projeto é compilado com o gcc por um script na raiz, um para cada sistema.

Importante: os arquivos `usuarios.csv`, `cotacoes.csv` e `extrato.csv` são lidos e gravados na pasta atual, por isso o programa deve ser executado na raiz do projeto.

Linux

Com o gcc instalado, dê permissão de execução ao script, compile e rode, sempre a partir da pasta raiz do projeto:

```
chmod +x build-linux.sh

./build-linux.sh

./Projeto_C_Exchange_Cripto
```

Windows

É necessário um gcc para Windows (MinGW-w64 ou MSYS2) e a pasta `bin` dele precisa estar na variável de ambiente `PATH`; caso contrário o `.bat` falha com "gcc não é reconhecido como um comando". Para conferir, abra um novo Prompt de Comando e execute `gcc --version`. Depois:

```
build-windows.bat
Projeto_C_Exchange_Cripto.exe
```

4. Dados

O módulo `src/dados` é a base do sistema. Na inicialização, `run.c` garante que os três arquivos existem (funções criar_arquivo_usuarios(), criar_arquivo_cotacoes() e criar_arquivo_extrato()) e carrega tudo para vetores em memória (`carregar_dados`). A partir daí, cada operação altera a memória e grava no arquivo na mesma hora, então nenhuma informação se perde ao fechar o programa. Os arquivos são CSV (texto separado por vírgulas), com uma linha de cabeçalho:

Estrutura dos Arquivos:

- **usuarios.csv:**
  - *Colunas:* `user_id`, `nome`, `CPF`, `password`, `saldo_real`, `saldo_btc`, `saldo_eth`, `saldo_xrp`
  - *Como é gravado:* Regravado por inteiro (`salvar_usuarios`) a cada mudança de saldo ou cadastro.
- **cotacoes.csv:**
  - *Colunas:* `moeda_id`, `valor`
  - *Como é gravado:* Regravado por inteiro (`salvar_cotacoes`); 0 = *BTC*, 1 = *ETH*, 2 = *XRP*.
- **extrato.csv:**
  - *Colunas:* `user_id`, `data`, `sinal`, `valor`, `ativo`, `cotacao`, `taxa`, `saldo_real`, `saldo_btc`, `saldo_eth`, `saldo_xrp`
  - *Como é gravado:* Só acrescenta uma linha por operação (`salvar_extrato_registro`); guarda os saldos depois da operação.

`dados.h`

Concentra as constantes, as estruturas e as declarações.

- Constantes: `MAX_USUARIOS` (100), `MAX_EXTRATO` (10.000), `NUM_MOEDAS` (3), os tamanhos de texto `TAM_*` (nome, CPF, senha, data, ativo), os índices `MOEDA_BTC/ETH/XRP` (0, 1, 2), os nomes dos arquivos e seus cabeçalhos, e `NAO_ENCONTRADO` (valor retornado pelas buscas quando não há resultado).
- Estruturas: `struct usuarios` (id, nome, CPF, senha, saldo em reais e vetor saldo_cripto[]), `struct cotacoes` (id da moeda e preço de 1 unidade em reais) e `struct extrato` (usuário, data, sinal +/-, valor, ativo, cotação, taxa e saldos após a operação).

`memoria.c`

- `lista_usuarios[]`, `lista_cotacoes[]`, `lista_extrato[]`
  - Vetores globais que guardam os dados em memória, com os contadores `total_usuarios` e `total_extrato`; também define `sigla_moeda[]` (BTC, ETH, XRP) e `nome_moeda[]` (Bitcoin, Ethereum, Ripple).
- `posicao_usuario_por_id(id)`
  - Procura o usuário pelo id e devolve a posição na lista (ou NAO_ENCONTRADO).
- `posicao_usuario_por_cpf(cpf)`
  - Mesma coisa da anterior, comparando o CPF com `strcmp`.
- `proximo_id_usuario()`
  - Devolve o maior id existente + 1, usado ao cadastrar.

`arquivos.c`

- `verificar_arquivo(nome)`
  - Tenta abrir o arquivo para leitura; retorna true se o arquivo existe, false se não existe.
- `criar_arquivo(nome, cab)`
  - Foi usada o tipo static pois essa função só deve ser (e efetivamente só é) usada dentro do arquivos.c, semelhante com oq é feito com public/private no

Java. Se o arquivo não existe, o arquivo é criado e o cabeçalho é escrito.

- `criar_arquivo_usuarios()`,`criar_arquivo_cotacoes()`, `criar_arquivo_extrato()`
  - Chamam `criar_arquivo` com o nome e o cabeçalho de cada CSV.

`carrega_dados.c`

- `cotacoes_padrao()`
  - Mais uma função privada, preenche as cotações iniciais na memória.
- `carregar_usuarios()`
  - Pula o cabeçalho e lê cada linha com `sscanf` para um usuário; para ao encontrar linha incompleta ou ao atingir o limite.
- `carregar_cotacoes()`
  - Começa pelas cotações padrão, sobrescreve com as do arquivo (ids e valores válidos) e, se faltar alguma, regrava o arquivo.
- `carregar_extrato()`
  - Lê cada linha do extrato para a lista, no mesmo formato de `carregar_usuarios`.
- `carregar_dados()`
  - Executa as três leituras (sempre todas) e devolve true se todas deram certo.

`salva_dados.c`

- `salvar_usuarios()`
  - Grava no `usuarios.csv` inteiro a partir dos dados em memória.
- `salvar_cotacoes()`
  - Grava no `cotacoes.csv` inteiro (valor com 6 casas).
- `salvar_extrato_registro(pos)`
  - Abre o extrato em modo append e acrescenta apenas a operação da posição `pos`.

5. Login

O módulo `src/login` cuida do cadastro e da lógica autenticação, recebendo os dados dos menus.

`login_cadastro.c`

- `so_digitos(texto, tamanho)`
  - Função privada, confere se o texto tem exatamente tamanho caracteres e se todos são dígitos (`isdigit`).
- `cpf_valido(cpf)`
  - Aceita somente 11 dígitos. Valida o formato (não calcula os dígitos verificadores).
- `senha_valida(senha)`
  - Aceita somente 6 dígitos, invalida demais senhas.
- `cadastrar(nome, cpf, senha)`
  - Valida CPF, senha e nome, rejeita CPF já cadastrado e verifica o limite de usuários;
  - Se tudo estiver certo, zera a struct (saldos iniciam em zero), gera o id com `proximo_id_usuario()` do outro módulo, copia os textos com limite de tamanho, troca vírgulas do nome por espaço (pra não quebrar o CSV) e chama `salvar_usuarios`;
  - Se a gravação falhar, desfaz o cadastro na memória.
  - Retorna um código `CAD_OK`, `CAD_CPF_INVALIDO`, `CAD_SENHA_INVALIDA`, `CAD_NOME_INVALIDO`, `CAD_CPF_EXISTENTE`, `CAD_LIMITE_USUARIOS` ou `CAD_ERRO_ARQUIVO`, que é usado pelo menu para informar o usuário sobre seu cadastro.
- `login(cpf, senha)`
  - Busca o usuário pelo CPF e compara a senha. Devolve o `user_id` se estiver correto ou 0 se falhar (os ids começam em 1, então 0 nunca é um usuário válido).
  - Depois do login, o `user_id` é passado a todas as funções do menu do usuário. Operações sensíveis (saldo, extrato, compra e venda) pedem a senha de novo, em `confirmar_senha`, no `submenu.c`.

6. Menu

O `run.c` chama `srand(time(NULL))` (para as cotações aleatórias), cria os arquivos, carrega os dados e repete `main_menu_login()` até ela devolver true (opção "Encerrar programa"). A pasta `src/menu` tem três arquivos:

`entrada.c`

Todo texto digitado pelo usuário passa por este arquivo. Ele existe porque o scanf sozinho tem problemas: deixa o Enter no buffer (a leitura seguinte "pula"), estoura vetores com entradas longas e, quando o teclado é fechado (EOF), retorna sem parar e prende o programa em laço infinito.

- `limpar_buffer()`
  - Lê caracteres até o `\n` (ou EOF), descartando o que sobrou na linha. É chamada depois de cada scanf.
- `fim_da_entrada()`
  - Função privada, chamada quando o teclado acaba (Ctrl+D no Linux, Ctrl+Z e Enter no Windows): avisa e encerra com `exit(0)`.
- `ler_numero_da_linha()`
  - Função privada, lê a linha inteira com `fgets` (128 bytes). Se a linha for maior que o buffer, descarta o

restante e devolve -1. Remove `\n` e `\r`, troca a vírgula decimal por ponto (aceita "10,50") e converte com `strtod`, conferindo `errno` e onde a conversão parou: só são toleradas sobras de espaços/tabs; qualquer outro caractere ("12abc") devolve -1.

- `ler_nome(nome)`
  - `scanf(" %63[^\n]")`: lê um texto com espaços, ignora espaços iniciais e limita a 63 caracteres (buffer de 64).
- `ler_palavra(palavra)`
  - scanf("%31s")`: lê uma palavra sem espaços, no máximo 31 caracteres. Usada para CPF e senha.
- `ler_opcao()`
  - Lê um inteiro com `scanf("%d")`. Devolve 0 se não for número ou for negativo; por isso as opções dos menus começam em 1 e o 0 serve como "inválido/cancelar".
- `ler_valor()`
  - Usa `ler_numero_da_linha` e arredonda para centavos com `arredondar_centavos`. Devolve -1 se inválido; quem chama trata valor ≤ 0 como "Valor inválido".

`menu_principal.c` e `submenu.c`

Menus simples associados as funções do projeto.

7. Moedas

O módulo `src/moedas` reúne as regras financeiras. Todas as operações devolvem um código `OP_OK`, `OP_USUARIO_INEXISTENTE`, `OP_VALOR_INVALIDO`, `OP_SALDO_INSUFICIENTE`, `OP_MOEDA_INVALIDA` ou `OP_ERRO_ARQUIVO`.

`compra_venda.c`

- `arredondar_centavos(v)`
  - Arredonda o valor para 2 casas (evita erros de ponto flutuante em dinheiro).
- `obter_taxa_compra/venda(tipo)`
  - Retorna a taxa da moeda.
- `valor_compra` / `valor_venda`
  - Compra: valor + taxa (quanto o usuário paga). Venda: valor - taxa (quanto recebe).
- `efetivar_compra(id, tipo, valor)`
  - O usuário informa o valor em reais; recebe valor / cotação em cripto e paga o valor mais a taxa. Recusa se o saldo em reais não cobrir o custo. Altera a memória, registra no extrato e grava `usuarios.csv`; se a gravação do extrato falhar, restaura uma cópia do usuário.
- `efetivar_venda(id, tipo, valor)`
  - Vende valor * cotação em cripto e credita o valor menos a taxa. Recusa se o usuário não tiver a quantidade (saldo nunca fica negativo); mesmo esquema de gravação e desfazer.

`cotacao.c`

- `gerar_numero_aleatorio()`
  - Devolve um número entre -0,05 e +0,05 (variação de -5% a +5%) a partir de `rand()`.
- `atualizar_cotacoes()`
  - Multiplica a cotação de cada moeda por (1 + variação) e grava `cotacoes.csv`.
- `exibir_cotacoes()`
  - Lista as três moedas numeradas com o preço atual em reais.

`saldo_extrato.c`

- `registrar_extrato(...)`
  - Preenche um registro com data/hora (`strftime`), sinal, valor, ativo, cotação, taxa e os saldos atuais; grava no arquivo e só então incrementa `total_extrato`.
- `depositar` / `sacar(id, valor)`
  - Somam ou subtraem reais (saque recusado se o saldo ficar negativo), registram no extrato (ativo REAL, cotação e taxa 0) e salvam.
- `exibir_dados_usuario(id)`
  - Função privada, mostra nome e CPF formatado (123.456.789-00).
- `exibir_saldo(id)`
  - Mostra os dados do usuário e os saldos em reais, Bitcoin, Ethereum e Ripple.
- `exibir_extrato(id)`
  - Percorre o extrato em memória e imprime apenas as operações do usuário; se não houver nenhuma, imprime uma mensagem na tela.